

Bash Scripting — Variabili, Condizioni, Loop

Modulo: 1A | **Corso:** Lab Amministrazione di Sistemi T

Prerequisiti: Moduli 0A e 0B (filesystem, pipe, redirect, filtri)

VM: `cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`

Cos'è uno script Bash

Finora hai usato i comandi uno alla volta nel terminale. Uno script Bash è un file di testo che contiene una sequenza di comandi: la shell li esegue in ordine, uno dopo l'altro, come se li stessi digitando tu.

Questo è il punto di partenza dell'automazione: invece di ripetere 20 comandi ogni volta, scrivi lo script una volta e lo esegui quando serve.

Creazione di uno script Bash

Il pattern heredoc

Per creare uno script direttamente dal terminale si usa il pattern cat con heredoc:

```
cat > nomeFile.sh << 'EOF'
#!/bin/bash
echo "ciao mondo, oggi è il $(date)"
EOF
```

Analisi riga per riga: - `cat > nomeFile.sh` — reindirige l'output di cat nel file `nomeFile.sh`, creandolo se non esiste - `<< 'EOF'` — dice alla shell di aspettarsi righe di testo fino a quando non incontra una riga contenente solo la parola EOF (il delimitatore può essere qualsiasi parola, per convenzione si usa EOF) - Tutto ciò che si scrive tra l'apertura e EOF finisce nel file esattamente come digitato

Lo shebang `#!/bin/bash`

La prima riga di ogni script deve essere lo shebang:

```
#!/bin/bash
```

Indica al sistema operativo quale interprete usare per eseguire il file. Senza di essa il sistema non saprebbe che si tratta di uno script Bash.

Rendere eseguibile ed eseguire

Come visto nel Modulo 0A, i file appena creati non hanno il permesso di esecuzione. Lo si aggiunge con:

```
chmod +x nomeFile.sh      # oppure: chmod 755 nomeFile.sh
./nomeFile.sh
```

Il prefisso ./ indica “esegui questo file nella directory corrente”.

Command substitution: \$()

La sintassi \$(comando) esegue un comando e inserisce il suo output inline nella stringa:

```
echo "oggi è il $(date)"
echo "sono l'utente $(whoami)"
```

Tutto ciò che è dentro \$() viene eseguito come comando e il suo output sostituisce l'intera espressione.

Variabili

Assegnazione e lettura

```
NOME="vagrant"
ETA=25
SALUTO="Benvenuto"

echo "$SALUTO, $NOME!"
echo "Hai $ETA anni."
echo "Il nome '$NOME' ha ${#NOME} caratteri."
```

Regole fondamentali: - Assegnazione: **VARIABILE=valore** — **senza spazi** intorno a = (con spazi Bash interpreta il primo token come comando) - Lettura: **\$VARIABILE** oppure **\${VARIABILE}** — le graffe sono obbligatorie in contesti ambigui (es. **\${NOME}file** — senza graffe Bash cercherebbe **\$NOMEFILE**) - **\${#VARIABILE}** restituisce la lunghezza della stringa - Per convenzione le variabili in Bash si scrivono in MAIUSCOLO

Variabili speciali

Variabile	Significato
\$0	Nome del file script

Variabile	Significato
\$1, \$2, \$3, ...	Il primo, secondo, terzo argomento passato all'esecuzione: ./script.sh arg1 arg2
\$#	Numero totale di argomenti passati
\$@	Tutti gli argomenti come lista

Esempio:

```
cat > argomenti.sh << 'EOF'
#!/bin/bash
echo "Nome dello script: $0"
echo "Primo argomento: $1"
echo "Secondo argomento: $2"
echo "Numero di argomenti: $#"
```

```
echo "Tutti gli argomenti: $@"
EOF
chmod +x argomenti.sh
./argomenti.sh ciao mondo
```

Output:

```
Nome dello script: ./argomenti.sh
Primo argomento: ciao
Secondo argomento: mondo
Numero di argomenti: 2
Tutti gli argomenti: ciao mondo
```

Variabili d'ambiente

Alcune variabili esistono già nel sistema senza doverle dichiarare:

```
echo $HOME      # /home/vagrant – home directory dell'utente corrente
echo $USER      # vagrant – nome utente corrente
echo $SHELL     # /bin/bash – shell in uso
echo $PATH      # lista di directory dove la shell cerca i programmi
```

\$PATH è particolarmente importante: quando digiti `ls`, la shell cerca il programma `ls` in ogni directory elencata in `$PATH`, nell'ordine. Se un programma non viene trovato, è perché la sua directory non è in `$PATH`.

Condizioni

Struttura if/then/else

```
cat > controllo_numero.sh << 'EOF'
#!/bin/bash
NUMERO=10

if [ $NUMERO -gt 5 ]; then
    echo "$NUMERO è maggiore di 5"
else
    echo "$NUMERO non è maggiore di 5"
fi
EOF
chmod +x controllo_numero.sh
./controllo_numero.sh
```

Output: 10 è maggiore di 5

La struttura generale è:

```
if [ condizione ]; then
    # comandi se vero
elif [ altra_condizione ]; then
    # comandi se seconda condizione vera
else
    # comandi se tutto falso
fi
```

fi chiude il blocco if (è “if” scritto al contrario — convenzione Bash).

Attenzione alla formattazione: gli spazi dentro le parentesi quadre [condizione] sono obbligatori. [\$NUMERO -gt 5] genera un errore; [\$NUMERO -gt 5] è corretto. Bash è rigido su questo.

Operatori di confronto

Per i **numeri** si usano flag letterali (non simboli matematici, che in Bash hanno altri significati):

Operatore	Significato
-eq	uguale (equal)
-ne	diverso (not equal)
-gt	maggiore di (greater than)
-lt	minore di (less than)
-ge	maggiore o uguale (greater or equal)
-le	minore o uguale (less or equal)

Per le **stringhe**:

Operatore	Significato
=	uguale
!=	diverso
-z "\$VAR"	stringa vuota (zero length) — usato per verificare argomenti mancanti
-n "\$VAR"	stringa non vuota

Per i **file**:

Operatore	Significato
-f file	esiste ed è un file regolare
-d path	esiste ed è una directory
-e path	esiste (file o directory, qualsiasi tipo)
-r file	esiste ed è leggibile
-w file	esiste ed è scrivibile
-x file	esiste ed è eseguibile

La negazione di qualsiasi condizione si ottiene con **!** prima della condizione:
[**!** -d "\$DIR"] è vero se \$DIR **non** è una directory.

Condizioni su file ed exit code

```
cat > controllo_file.sh << 'EOF'
#!/bin/bash
FILE=$1

if [ -z "$FILE" ]; then
    echo "Errore: devi specificare un file come argomento"
    exit 1
fi

if [ -f "$FILE" ]; then
    echo "$FILE esiste ed è un file"
elif [ -d "$FILE" ]; then
    echo "$FILE esiste ed è una directory"
else
    echo "$FILE non esiste"
fi
EOF
chmod +x controllo_file.sh
```

```
./controllo_file.sh /etc/hostname
./controllo_file.sh /etc
./controllo_file.sh /file_inesistente
./controllo_file.sh
```

Output (una riga per ogni invocazione):

```
/etc/hostname esiste ed è un file
/etc esiste ed è una directory
/file_inesistente non esiste
Errore: devi specificare un file come argomento
```

Exit code: exit 1 termina lo script con codice di uscita 1 (errore). Per convenzione exit 0 indica successo, qualsiasi altro valore indica errore. I codici di uscita sono fondamentali nell'automazione: permettono ad uno script di sapere se un altro script o comando è andato a buon fine, e di reagire di conseguenza.

Loop

Loop for — su lista

```
cat > loop_for.sh << 'EOF'
#!/bin/bash
for FRUTTO in mela pera banana arancia; do
    echo "Frutto: $FRUTTO"
done
EOF
chmod +x loop_for.sh
./loop_for.sh
```

Output: quattro righe, una per ogni frutto.

Struttura:

```
for VARIABILE in elemento1 elemento2 elemento3; do
    # comandi — $VARIABILE vale l'elemento corrente
done
```

Risposta al dubbio — la variabile del for non si dichiara prima: la variabile del loop (FRUTTO in questo caso) viene creata direttamente dall'istruzione for, non occorre dichiararla in anticipo. Bash le assegna ogni elemento della lista in sequenza. Il nome è arbitrario: avrei potuto scrivere for X in ... o for QUALSIASI_NOME in ... — l'importante è usare lo stesso nome dentro il loop con \$. La variabile rimane accessibile anche dopo done, con il valore dell'ultimo elemento iterato.

Loop for — su file con glob

```
cat > loop_file.sh << 'EOF'
#!/bin/bash
for FILE in /etc/*.conf; do
    echo "Trovato file di configurazione: $FILE"
done
EOF
chmod +x loop_file.sh
./loop_file.sh
```

Output: una riga per ogni file .conf in /etc.

Risposta al dubbio — FILE non è una parola riservata: FILE è semplicemente il nome che ho scelto per la variabile del loop. Non è un termine riservato di Bash — avrei potuto scrivere `for ELEMENTO in /etc/*.conf` o `for X in /etc/*.conf` ottenendo lo stesso risultato. La shell espande *.conf nella lista di tutti i file corrispondenti al pattern *prima* di passarla al loop, e FILE assume il percorso completo di ognuno (/etc/adduser.conf, /etc/debconf.conf, ecc.) ad ogni iterazione.

Loop for — numerico con \$(seq)

```
cat > loop_numerico.sh << 'EOF'
#!/bin/bash
for I in $(seq 1 5); do
    echo "Iterazione numero: $I"
done
EOF
chmod +x loop_numerico.sh
./loop_numerico.sh
```

seq 1 5 genera la sequenza 1 2 3 4 5. \$(seq 1 5) la inserisce come lista nel for tramite command substitution.

Altri comandi utili con \$():

Espressione	Cosa produce
\$(seq 1 10)	numeri da 1 a 10
\$(seq 0 2 10)	numeri da 0 a 10 a passi di 2: 0 2 4 6 8 10
\$(ls /etc)	lista dei file in /etc (uno per riga)
\$(cat file.txt)	contenuto di un file riga per riga
\$(date +%Y-%m-%d)	data corrente in formato YYYY-MM-DD
\$(hostname)	nome della macchina
\$(wc -l < file.txt)	numero di righe di un file
\$(whoami)	nome dell'utente corrente

Il pattern for RIGA in `$(cat file.txt)` è molto usato in script di automazione per elaborare ogni riga di un file.

Loop while

```
cat > loop_while.sh << 'EOF'
#!/bin/bash
CONTATORE=1
while [ $CONTATORE -le 5 ]; do
    echo "Contatore: $CONTATORE"
    CONTATORE=$((CONTATORE + 1))
done
EOF
chmod +x loop_while.sh
./loop_while.sh
```

Output: cinque righe con il contatore da 1 a 5.

`$((espressione))` è la sintassi per l'**aritmetica** in Bash — tutto ciò che è nelle doppie parentesi viene interpretato come operazione matematica. `CONTATORE=$((CONTATORE + 1))` incrementa la variabile di 1 ad ogni iterazione.

Differenza for vs while: - for si usa quando la lista di elementi è nota in anticipo (file, numeri, argomenti) - while si usa quando si ripete finché una condizione è vera (es. aspettare che un servizio sia attivo, contare fino a un valore)

Script completo — analizza_dir.sh

```
cat > analizza_dir.sh << 'EOF'
#!/bin/bash
DIRECTORY=$1

if [ -z "$DIRECTORY" ]; then
    echo "Uso: $0 <directory>"
    exit 1
fi

if [ ! -d "$DIRECTORY" ]; then
    echo "ERRORE: $DIRECTORY non è una directory"
    exit 1
fi

echo "=== Analisi di: $DIRECTORY ==="
```



```

CONTATORE_FILE=0
CONTATORE_DIR=0

for ELEMENTO in "$DIRECTORY"/*; do
    if [ -f "$ELEMENTO" ]; then
        CONTATORE_FILE=$((CONTATORE_FILE + 1))
    elif [ -d "$ELEMENTO" ]; then
        CONTATORE_DIR=$((CONTATORE_DIR + 1))
    fi
done

echo "File trovati: $CONTATORE_FILE"
echo "Directory trovate: $CONTATORE_DIR"
echo "Totale elementi: $((CONTATORE_FILE + CONTATORE_DIR))"
EOF
chmod +x analizza_dir.sh

./analizza_dir.sh /etc
./analizza_dir.sh /var/log
./analizza_dir.sh

```

Errore da evitare — la logica della condizione negata: nella tua versione avevi scritto:

```

if [ -d "$DIRECTORY" ]; then
    echo "ERRORE: $DIRECTORY non è una directory"    # ← sbagliato

```

Questo stampa l'errore esattamente quando \$DIRECTORY è una directory — l'opposto di quello che vuoi. La versione corretta usa ! per negare:

```

if [ ! -d "$DIRECTORY" ]; then
    echo "ERRORE: $DIRECTORY non è una directory"    # ← corretto: errore se NON

```

Questo pattern — verificare una condizione e uscire con errore se non è soddisfatta — si chiama **guard clause** (clausola di guardia). Lo vedrai ovunque negli script reali: prima di procedere, verifica che le precondizioni siano rispettate.

Esercizio in autonomia — cerca_file.sh

Tua versione (con analisi degli errori)

```
#!/bin/bash
```

```

NOMEFILE=$1
NOMEDIR=$2

```

```

if [ -e "$NOMEFILE" ]; then
    # (branch then vuoto)
else
    echo "ERRORE: $1 non è un file o un path"
fi

if [ -d "$NOMEDIR" ]; then
    # (branch then vuoto)
else
    echo "ERRORE: $2 non è una directory"
fi

for ELEMENTO in "$2"$1; do
    if [ -r "$ELEMENTO" ]; then
        echo "il file $ELEMENTO è leggibile"
    else
        echo "il file $ELEMENTO non è leggibile"
    fi
    if [ -w "$ELEMENTO" ]; then
        echo "il file $ELEMENTO è scrivibile"
    else
        echo "il file $ELEMENTO non è scrivibile"
    fi
fi

```

Errori presenti:

1. `#!/bin/bash` — manca il `#`. Senza shebang lo script potrebbe essere eseguito dalla shell sbagliata o produrre errori incomprensibili.
2. **Logica invertita nel secondo if:** `if [ -d "$NOMEDIR" ]` è vero quando `$NOMEDIR` è una directory — quindi il branch `else` (che stampa l'errore) scatta quando non lo è, ma il branch `then` è vuoto e non fa nulla. Funziona per caso sul ramo dell'errore, ma la struttura è ambigua. La forma canonica è `if [ ! -d "$NOMEDIR" ]; then echo "ERRORE" ; exit 1; fi`.
3. **Il primo if controlla la cosa sbagliata:** `[ -e "$NOMEFILE" ]` verifica se esiste un file chiamato esattamente `$NOMEFILE` (es. `*.sh`) — ma `$NOMEFILE` è un *pattern*, non un percorso. La verifica corretta per un argomento mancante è `[ -z "$NOMEFILE" ]`.
4. **Il loop for ELEMENTO in "\$2"\$1:** `"$2"` è quotato (trattato come stringa letterale), ma `$1` è concatenato direttamente senza separatore. Se `$2=/home/vagrant` e `$1=*.sh`, il risultato è `/home/vagrant*.sh` — manca lo `/`. La forma corretta è `"$NOMEDIR"/$NOMEFILE` (dove `$NOMEFILE` non va quotato, altrimenti il glob `*` non viene espanso).
5. **done mancante:** il loop `for` non viene chiuso. Bash darebbe un errore di

sintassi.

L'iniziativa di aggiungere -w (controllo scrittura) è corretta e utile — non era richiesta ma è una buona estensione della logica.

Versione corretta

```
cat > cerca_file.sh << 'EOF'
#!/bin/bash

NOMEFILE=$1
NOMEDIR=$2

if [ -z "$NOMEFILE" ] || [ -z "$NOMEDIR" ]; then
    echo "Uso: $0 <pattern> <directory>"
    exit 1
fi

if [ ! -d "$NOMEDIR" ]; then
    echo "ERRORE: $NOMEDIR non è una directory"
    exit 1
fi

for ELEMENTO in "$NOMEDIR"/$NOMEFILE; do
    if [ -f "$ELEMENTO" ]; then
        if [ -r "$ELEMENTO" ]; then
            echo "$ELEMENTO è leggibile"
        else
            echo "$ELEMENTO non è leggibile"
        fi
        if [ -w "$ELEMENTO" ]; then
            echo "$ELEMENTO è scrivibile"
        else
            echo "$ELEMENTO non è scrivibile"
        fi
    fi
done
EOF
chmod +x cerca_file.sh
```

```
./cerca_file.sh "*.conf" /etc
./cerca_file.sh "*.sh" ~/Lab1A
```

Differenze chiave rispetto alla tua versione: - Argomenti mancanti verificati con -z su entrambi - Condizioni di guardia con ! e exit 1 — si esce subito in caso di errore, non si continua - "\$NOMEDIR"/\$NOMEFILE con \$NOMEFILE non

quotato — il glob viene espanso correttamente - `if [ -f "$ELEMENTO" ]` prima dei controlli permessi — si evita di stampare permessi su directory o file inesistenti - `done` chiude il loop

Riepilogo sintattico

Sintassi	Funzione
<code>VAR=valore</code>	Assegna variabile (senza spazi intorno a =)
<code>\$VAR / \${VAR}</code>	Legge variabile
<code>\${#VAR}</code>	Lunghezza della stringa
<code>\$1, \$2, \$#, \$@</code>	Argomenti dello script
<code>\$(comando)</code>	Command substitution — inserisce output del comando
<code>\$((espressione))</code>	Aritmetica intera
<code>if [cond]; then ... fi</code>	Condizionale
<code>[! condizione]</code>	Negazione della condizione
<code>for VAR in lista; do ... done</code>	Loop su lista/glob
<code>while [cond]; do ... done</code>	Loop con condizione
<code>exit 0 / exit 1</code>	Termina con successo / errore
<code>-gt, -lt, -eq, -le</code>	Confronto numerico
<code>-z, -n</code>	Stringa vuota / non vuota
<code>-f, -d, -e, -r, -w, -x</code>	Test su file/directory

Connessione con Security

Gli script Bash sono la base di quasi tutti i tool di automazione offensiva e difensiva: - Script di enumerazione che ciclan con `for` su una lista di host o porte - Script di monitoring che controllano con `while` se un servizio è ancora attivo - Script di risposta agli incidenti che verificano con `if` la presenza di file sospetti - I tool come Nmap e Metasploit usano internamente script con queste stesse strutture